Polymorphism Revisited

Today, we revisit polymorphism in the context of inheritance and interfaces. As class hierarchies get more complex, it is important to be able to trace what happens when we call a method from a variable, especially if that variable's declared type and instantiated type are different.

Content Learning Targets

After completing this activity, you should be able to say:

- I can identify the declared type and instantiated type of a variable.
- I can trace polymorphic method calls in a class hierarchy.
- I can identify, and distinguish, compile-time and runtime errors in polymorphic method calls.

Process Skill Goals

During the activity, you should make progress toward:

- Predicting code output before running it and identifying patterns. (Critical Thinking)
- Forming shared understanding via team discussion. (Teamwork)

Facilitation Notes


Copyright © 2026 Ian Ludden, based on prior work of Mayfield et al. This work is under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Part 1 Tracing Polymorphic Method Calls

When we call a method from a variable whose type is in a complex class hierarchy, it can be tricky to sort out what actually happens. We start today with the following polymorphic method call "puzzles".

```
class A {
    public void print() {
        System.out.println("A");
    }
}

class B extends A {
    public void print() {
        System.out.println("B");
    }
}

class C extends B {
    public void print() {
        System.out.println("C");
    }
    public void printC() {
        print();
    }
}

class D extends C {
    public void print() {
        System.out.println("D");
    }
}
```

1. Using the space on the right, draw the UML diagram for these classes.

PlantUML link

2. The superclass of `A` is... `Object`

3. What method(s) is/are available to an object of type `D`?

`print()` (overridden) and `printC()` (inherited from `C`), as well as all methods inherited from `Object`

4. For each of the following code snippets, write down what is printed when the code is run. If there is a compile-time error, write "compile error". If there is a runtime error, write "runtime error". (If you're not sure, make your best guess.)
Some variables are reused, such as `x` from the first snippet. If an error happens in one snippet, assume it is commented out before running the next snippet.

a)
```
A x = new D();
x.print();
```

D

b)
```
D x2 = (D)x;
x2.print();
```

D

c)
```
x2.printC();
```

D

d)
```
B x3 = (B)x;
x3.print();
```

D

e)
```
x3.printC();
```

compile error (`B has no method printC()`)

f)
```
C x4 = new C();
x4.printC();
```

C

g)
```
D x5 = (D)x4;
x5.print();
```

runtime error (`ClassCastException`)

5. In `src/polymorphicMethods`, check your work by uncommenting each code snippet. Were there any surprises?

You may be surprised to see that casting `x` to a `B` does not change the method that gets called; we still use the `D` version of `print()`.

6. In your team, try to come up with a set of rules for determining which method ***actually*** gets called when a method is invoked on a polymorphic variable.

See the five-step process in the slides.

7. Individually, complete the Polymorphic Tracing Practice activity in Moodle.